

CI/CD جزوه مرور

تمام مطالبی که در این گفتوگو تا اینجا پوشش داده شد

.NET + GitLab CI/CD + Docker concepts :تمرکز

هایی که در طول آموزش مطرح شد. این فایل فقط مطالب گفته شده در همین Syntax هدف جزوه :مرور سریع مفاهیم، تفاوتها و گفتوگو را جمع بندی می کند.

1. CI/CD چیست؟

برای خودکارسازی فرایند تحویل نرم افزار استفاده می شود. به جای اینکه توسعه دهنده بعد از هر تغییر همه مراحل را CI/CD مفهوم این مراحل را طبق تعریف تیم اجرا می کند Pipeline، دستی انجام دهد.

```
Developer
|
git push
v
GitLab Pipeline
|
+--> Restore
+--> Build
+--> Test
+--> Publish / Package
+--> Deploy
```

CI یا Continuous Integration

و Build، Restore شدن تغییرات، سیستم به صورت خودکار بررسی می کند که کد قابل Merge یا Push بعد از CI در این است که تغییرات ناسالم هر چه زودتر شناسایی شوند CI هست یا نه. هدف اصلی Test

```
dotnet restore
dotnet build
dotnet test
```

CD یا Continuous Delivery / Deployment

Deploy هدف Environment نسخه ای که مراحل اعتبارسنجی را پاس کرده برای انتشار آماده می شود یا روی CD در بخش می شود.

مفهوم	توضیح
Continuous Delivery	به Deploy نسخه تا مرحله آماده انتشار پیش می رود، اما معمولاً نیاز به تأیید دستی دارد Production.
Continuous Deployment	نیز Production به Deploy، موفق باشد Pipeline اگر به صورت خودکار انجام می شود.

Continuous انتشار آماده است ولی ممکن است دستی تأیید شود؛ در Continuous Delivery **حفظی مهم** : در Deployment انتشار هم خودکار انجام می‌شود.

2. Pipeline، Stage و Job

Job و Stage و Pipeline: ساختار اصلی از سه مفهوم تشکیل می‌شود GitLab CI در

مفهوم	معنی
Pipeline	از شروع تا پایان CI/CD کل فرایند
Stage	build، test یا deploy مانند Pipeline یک مرحله منطقی از
Job	اجرا می‌شود Stage یک کار مشخص که داخل یک

```
stages:
  - build
  - test
  - deploy

build-job:
  stage: build
  script:
    - dotnet build

unit-test:
  stage: test
  script:
    - dotnet test
```

کافی و نبود وابستگی خاص، می‌توانند به‌صورت موازی اجرا Runner باشند، در صورت وجود Stage در یک Job اگر چند تست باشند Stage جدا در Job می‌توانند دو Integration Test و Unit Test شوند. برای مثال

```
+++> Unit Test -----+
Build -----+          +--> Deploy
+++> Integration Test -+
```

3. Fail Fast

Test نباید بدون دلیل به مراحل بعدی ادامه دهد. اگر Pipeline، یعنی وقتی یک مرحله مهم شکست می‌خورد Fail Fast اصل Test نباید انجام شود Deploy شکست بخورد، معمولاً

```
Build      OK
Unit Test   OK
Integration Test FAILED
Deploy      SKIPPED
```

4. GitLab Runner

GitLab Pipeline را واقعاً اجرا می‌کند Pipeline های Job یا محیطی است که Agent همان Runner را مدیریت می‌کند، ولی اجرای دستورها روی Runner می‌شود.

```
GitLab
|
v
Runner
|
+--> dotnet restore
+--> dotnet build
+--> dotnet test
```

یک Runner می‌تواند روی Linux Server، Virtual Machine، Docker Container یا Kubernetes Pod باشد.

dotnet در دسترس نباشد، دستور dotnet خطا می‌خورد. برای مثال اگر Job، ابزار لازم را نداشته باشد Runner اگر قابل اجرا نیست build.

5. Job برای محیط Docker Image استفاده از

یک Job باشد، می‌توان برای Runner قابل تکرار و مستقل از نرم‌افزارهای نصب‌شده روی Job برای اینکه محیط اجرای Docker Image مشخص کرد.

```
build:
  stage: build
  image: mcr.microsoft.com/dotnet/sdk:8.0
  script:
    - dotnet restore
    - dotnet build
```

موردنیاز را از قبل دارد NET SDK. را در محیطی اجرا می‌کند که Job Runner در این حالت.

6. تفاوت dotnet build و dotnet publish

کاربرد	دستور
است Build می‌کند و بررسی می‌کند کد قابل Compile پروژه را	dotnet build
ها، DLL تولید می‌کند؛ شامل Deploy خروجی آماده اجرا و dependencyهای موردنیاز اجرا	dotnet publish

انجام Build هم انجام می‌دهد. بنابراین اگر قبلاً Build به صورت پیش فرض خودش dotnet publish نکته مهم این است که غیرضروری تکرار نشود Build طوری باشد که Pipeline شده باشد، باید طراحی.

```
stages:
  - build
  - test
  - publish
  - deploy
```

```

build:
  stage: build
  script:
    - dotnet restore
    - dotnet build -c Release --no-restore

test:
  stage: test
  script:
    - dotnet test -c Release --no-build

publish:
  stage: publish
  script:
    - dotnet publish MyApi/MyApi.csproj -c Release -o publish

```

Stage از نظر اجرا الزاماً اشتباه نیست، چون نام build ی با نام Stage در dotnet publish نکته طراحی: قرار دادن publish خوانایی Pipeline مربوط به Job یا Stage قراردادی است؛ اما جدا کردن را بهتر می‌کند.

7. Artifact

های بعدی یا کاربر بتوانند از آن استفاده کنند. برای مثال پوشه Job است تا Job برای نگهداشتن خروجی یک Artifact است. Deploy خروجی قابل publish.

```

build:
  stage: build
  script:
    - dotnet publish -c Release -o publish
  artifacts:
    paths:
      - publish/

```

باشند Artifact می‌توانند test results یا build output، publish، خروجی‌هایی مثل فایل‌های.

8. Artifact و تفاوت آن با Cache

های Dependency استفاده می‌شود. برای مثال داتلود Pipeline برای کاهش کارهای تکراری و سرعت دادن به Cache استفاده کند Cache های متوالی می‌تواند از Pipeline در NuGet.

Artifact	Cache
های بعدی نگه می‌دارد Job را برای Job خروجی یک.	برای جلوگیری از داتلود یا کار تکراری استفاده می‌شود.
مثال: test-results یا publish/.	مثال: NuGet packages یا dependency folders.

کاهش کار تکراری = Cache؛ Job خروجی = Artifact: خلاصه

9. expire_in برای Artifact

Artifact می‌تواند GitLab، استفاده کرد. بعد از پایان این مدت expire_in می‌توان از Artifact برای تعیین مدت نگهداری را پاک کند.

```
artifacts:
  paths:
    - publish/
  expire_in: 2 days
```

باشند week یا 1 hours نمونه‌های دیگر می‌توانند 3

10. rules

شود یا اجرا شود. یکی از کاربردهای رایج، محدود Pipeline تحت چه شرطی وارد Job می‌توان مشخص کرد یک rules با اصلی است Branch به Deploy کردن.

```
deploy:
  stage: deploy
  script:
    - echo "Deploying..."
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

هم استفاده شود Merge Request یا Tag نیست و می‌تواند برای شرط‌های دیگر مانند Branch فقط برای rules قابلیت

11. when: manual

به Deploy استفاده می‌کنیم. این الگو برای when: manual فقط با اقدام دستی اجرا شود، از Job اگر بخواهیم یک Production بسیار رایج است.

```
deploy-production:
  stage: deploy
  script:
    - echo "Deploy to production"
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
  when: manual
```

نیاز به اقدام Job می‌گوید اجرای when: manual شود؛ Pipeline در چه شرایطی وارد Job تعیین می‌کند rules: تفاوت دستی دارد.

12. needs

موردنیازش شروع شود و Job می‌تواند به‌محض تمام شدن Job ها را تعریف می‌کنیم. یک Job وابستگی مستقیم بین needs با قبلی بماند Stage های Job لازم نیست همیشه منتظر تمام

```
build:
  stage: build
  script:
    - dotnet build
```

```
test:
  stage: test
  needs:
    - build
  script:
    - dotnet test
```

stages	needs
را مشخص می‌کند Pipeline ترتیب کلی مراحل	ها را مشخص می‌کند Job وابستگی مستقیم بین
را شکل می‌دهد Stage-by-Stage اجرای پیش‌فرض	قبل از Stage ممکن را بدون انتظار برای کل Job می‌تواند شروع کند.

ها را مشخص می‌کند Job وابستگی و ترتیب اجرای needs داده و خروجی را نگه می‌دارد؛ **Artifact:** تفاوت با

13. allow_failure

استفاده کرد `allow_failure: true` را متوقف کند، می‌توان از Pipeline نباید کل Job اگر شکست یک

```
security-scan:
  stage: test
  script:
    - run-security-scan
  allow_failure: true
```

های حیاتی بهتر Job کند. برای Block را Delivery هایی کاربرد دارد که شکست آنها فعلاً نباید Check این قابلیت برای است بدون دلیل از آن استفاده نشود.

14. before_script و after_script

before_script

یا اجرای Environment اجرا می‌شوند و برای آماده‌سازی Job اصلی script قبل از before_script دستورهای دستورات مقدماتی مناسب‌اند.

```
build:
  stage: build
  before_script:
    - echo "Preparing environment"
    - dotnet --info
  script:
    - dotnet restore
    - dotnet build
```

after_script

ها یا پاک کردن فایل های موقت Log جمع کردن، Cleanup اجرا می شوند و برای script بعد از after_script دستورهای برای کارهای پاکسازی اجرا می شود after_script، شکست بخورد script مناسباند. معمولاً حتی اگر

```
test:
  stage: test
  script:
    - dotnet test
  after_script:
    - echo "Cleaning temporary files"
    - rm -rf temp/
```

```
before_script
  |
  v
script
  |
  v
after_script
```

15. extends و Template های مشترک

از آن extends های دیگر با Job ساخت و Template ها می توان یک Job برای جلوگیری از تکرار تنظیمات مشترک بین استفاده کنند.

```
.common-dotnet:
  image: mcr.microsoft.com/dotnet/sdk:8.0
  before_script:
    - dotnet --info

build:
  extends: .common-dotnet
  stage: build
  script:
    - dotnet build

test:
  extends: .common-dotnet
  stage: test
  script:
    - dotnet test
```

ها است Job برای استفاده مجدد از تنظیمات مشترک بین extends: خلاصه

16. include

آن‌ها را وارد include تقسیم کرد و با YAML را در چند فایل Pipeline بزرگ شود، می‌توان .gitlab-ci.yml. اگر فایل Pipeline اصلی کرد Pipeline.

```
include:
- local: 'ci/build.yml'
- local: 'ci/test.yml'
- local: 'ci/deploy.yml'
```

```
.gitlab-ci.yml
ci/
  build.yml
  test.yml
  deploy.yml
```

بین چند فایل Pipeline برای تقسیم include ها است؛ Job برای اشتراک تنظیمات بین extends: extends تفاوت با YAML استفاده می‌شود.

17. GitLab CI/CD Variables و Secrets

نوشته شوند. بهتر .gitlab-ci.yml نباید مستقیم داخل Connection String و Token، Password مقادیر حساس مثل Variable فقط نام Pipeline تعریف شوند و GitLab CI/CD Variables است در.

```
script:
- echo "$DB_PASSWORD"
```

استفاده می‌شود Protected و Masked ها معمولاً از قابلیت‌های Secret برای.

معنی	نوع
مخفی می‌کند Pipeline های Log را در Variable مقدار	Masked Variable
Protected های Tag ها یا Branch را به Variable استفاده از محدود می‌کند.	Protected Variable

فعال Protected و Masked معمولاً منطقی است هر دو ویژگی Production مربوط به Password یا Token برای باشند.

18. Environment در CI/CD

Development، Staging و Production وجود دارد. نمونه‌های رایج Environment در پروژه واقعی معمولاً چند هستند.

کاربرد	Environment
توسعه و تست اولیه	Development
تست نهایی قبل از انتشار	Staging
محیط واقعی کاربران	Production

```

deploy-staging:
  stage: deploy
  script:
    - echo "Deploy to staging"
  environment:
    name: staging

deploy-production:
  stage: deploy
  script:
    - echo "Deploy to production"
  environment:
    name: production

```

ای روی کدام Pipeline ها را نگه دارد و مشخص باشد چه Deploy می‌تواند تاریخچه GitLab، Environment، با تعریف Environment Deploy شده است.

19. Environment-specific Variables

در DB_SERVER یا API_URL های مختلف مقدار متفاوتی دارد. برای مثال Environment در Variable گاهی یک Staging و Production نام همان نام Scope را نگه داشت ولی Variable یکسان نیست. می‌توان همان نام Environment متفاوت تعریف کرد.

```

API_URL
staging -> https://staging.example.com
production -> https://api.example.com

```

Environment مقدار مناسب GitLab استفاده کند، اما \$API_URL از یک نام ثابت مثل Pipeline این کار باعث می‌شود قرار دهد Job را در اختیار.

20. Flow تا اینجا مفهومی کامل

```

Developer
|
| git push
v
GitLab Repository
|
v
Pipeline
|
+--> Restore
+--> Build
+--> Test
+--> Publish
|   |
|   +--> Artifact
|

```

```
+--> Docker / Package step
|
+--> Deploy to Environment
|
+--> Development
+--> Staging
+--> Production
```

21. Pipeline نمونه جمع‌بندی

باید بر Production نمونه زیر بسیاری از مفاهیمی را که تا اینجا گفته شد کنار هم نشان می‌دهد. این مثال آموزشی است و برای واقعی تنظیم شود Runner اساس پروژه و

```
stages:
- build
- test
- publish
- deploy

.common-dotnet:
  image: mcr.microsoft.com/dotnet/sdk:8.0
  before_script:
    - dotnet --info

build:
  extends: .common-dotnet
  stage: build
  script:
    - dotnet restore
    - dotnet build -c Release --no-restore

test:
  extends: .common-dotnet
  stage: test
  needs:
    - build
  script:
    - dotnet test -c Release --no-build

publish:
  extends: .common-dotnet
  stage: publish
  needs:
    - test
  script:
    - dotnet publish MyApi/MyApi.csproj -c Release -o publish
  artifacts:
    paths:
      - publish/
    expire_in: 2 days

deploy-production:
```

```

stage: deploy
needs:
  - publish
script:
  - echo "Deploy to production"
environment:
  name: production
rules:
  - if: '$CI_COMMIT_BRANCH == "main"'
when: manual

```

22. مرور خیلی سریع

موضوع	یک خطی
CI	و اعتبار سنجی تغییرات Restore، Build، Test
Continuous Delivery	می تواند دستی Deploy Production نسخه آماده انتشار است و باشد.
Continuous Deployment	هم خودکار است Deploy Production
Pipeline	CI/CD کل فرایند
Stage	Pipeline مرحله منطقی
Job	کار مشخصی که اجرا می شود
Runner	Job اجرای Agent/محیط
Artifact	Job نگهداری خروجی
Cache	کاهش کار و دانلود تکراری
rules	Job شرط ورود/اجرای
when: manual	Job اجرای دستی
needs	Job وابستگی مستقیم بین
allow_failure	Job با وجود شکست Pipeline اجازه ادامه
before_script	script دستورهای قبل از
after_script	Cleanup مناسب، script دستورهای بعد از
extends	اشتراک تنظیمات مشترک
include	YAML در چند فایل Pipeline تقسیم
expire_in	Artifact مدت نگهداری
Masked Variable	Log مخفی کردن مقدار در
Protected Variable	Protected Ref های Variable به محدود کردن
Environment	production یا staging مانند Deploy محیط

23. مطالبی که در ادامه مسیر باقی مانده اند

: عملی کامل، موضوعات زیر هنوز باید عمیق تر آموزش داده شوند Pipeline در گفت و گو اشاره شد که برای رسیدن به یک

- GitLab در Cache تنظیم عملی
- dependencies و artifacts و ارتباط دقیق آن با
- Pipeline داخل Docker Image ساخت
- Registry به Docker Image کردن Push
- Environment واقعی به Deploy
- Merge Request و Tag، Branch پیشرفته تر برای rules
- ها و Secret مدیریت Variable واقعی
- Deploy در صورت خراب شدن Rollback

- Deploy از صفر تا .NET. کامل Pipeline ساخت یک
- CI/CD. مرور سؤالات مصاحبه‌ای

24. پاسخ‌های کوتاه مناسب مرور و مصاحبه

برای مرور سریع، این جواب‌های کوتاه را به خاطر بسپار:

- در آن اجرا می‌شوند Pipeline های Job محیطی است که Runner
- های بعدی نگه می‌دارد Job را برای Job خروجی یک Artifact
- ها استفاده می‌شود NuGet Package برای جلوگیری از تکرار کارهایی مثل دانلود Cache
- است Stage کار واقعی داخل Job را مشخص می‌کند و Pipeline مراحل کلی Stage
- می‌توانند موازی اجرا شوند Runner در صورت فراهم بودن Stage های یک Job
- استفاده می‌شود Job برای اجرای دستی when: manual و Job برای شرطی کردن rules
- وابسته است Job مستقیماً به کدام Job مشخص می‌کند یک needs
- تولید می‌کند Deploy خروجی مناسب dotnet publish است و Compile برای dotnet build
- های محافظت‌شده محدود می‌کند Ref استفاده از آن را به Protected پنهان می‌کند و Log را در Secret مقدار Masked